4 Implementation

The main goal of this system is to decrypt the encrypted program segments at run-time. The patch was compiled on a Debian 13 kernel, using the kernel patch version 6.12.58. The main topics provided in this section are the fol-lowing: design decisions, program preparation, process marking in the kernel, memory decryption implementation, and asymmetric key encryption

4.1 Design Decisions

A couple of design decisions have to be made beforehand to set a clear foundation for the working patch. A decision was made on the type of executable that will be used. A statically linked executable was chosen because the unrelocated symbol dependencies on shared library data at run-time would introduce unwanted complexity. The chosen encryption and decryption algorithm is the XOR cipher [10], because asymmetric key encryption changes the length of the instructions and data used by the program, leading to a reduced page size, which overcomplicates the actual implementation.

The chosen program segment for performing encryption and decryption is the text segment, because the complexity in decrypting the data or the text segment is the same, since their virtual address space is both predetermined at link-time and is available to be retrieved from the process control block structure at run-time. The heap segment was not chosen since the data inside the heap is only available at run-time. The stack was not chosen since the data is not yet available at link-time; it will be fetched from the text segment and pushed onto the stack at run-time.

4.2 Program Preparation

Program preparation is divided into two parts: marking and encryption. In order to test the kernel patch, a server program written in the `C` language was chosen. The server program listens on a specific port and infinitely waits for connections. In particular, there is no difference in what kind of program will be chosen, as long as it is statically compiled, the kernel patch should work on all programs. The encryption process is divided into two stages: the marking of the program and the text segment encryption.

The marking of the program is implemented in the Python programming language by using the `pwn` library. In order to identify the ELF file, the kernel checks each ELF file's magic numbers; this mechanism is exploited in order to change the fourth byte to the letter `C` as shown in Figure 6. This will allow the kernel to identify the special binary file once the program starts loading into memory.

The decision was made to encrypt only the text section. The text section of the server program

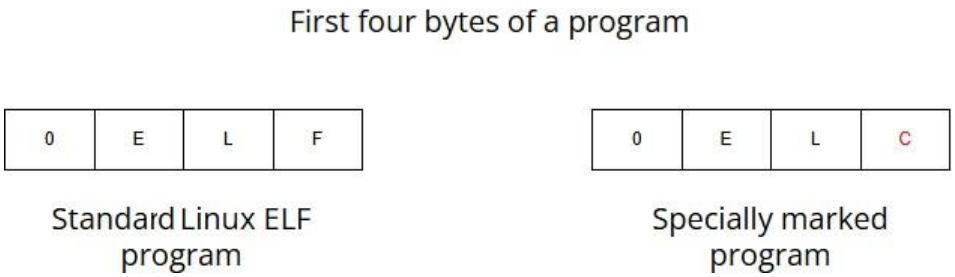

Figure 6. Changing the 3rd Byte of a Program to the Letter `C`.

is 119 pages, which is approximately 490KB, which is more than half of the entire program (most of the text code is filled with `libc.so` library code). Encryption was implemented by a program written in the `C` programming language. The chosen program is read byte by byte, and from the desired offset, in this case from the start of the text segment, till the end of the text segment, each byte is encrypted and written to the ELF file.

4.3 Process Marking in the Kernel

For the kernel to detect a process that is unique from all other processes, two things must be implemented in the kernel source code. First, an additional integer is added to each newly created process control block. This integer value will be used to mark the currently running process as special, setting its value equal to 1 (because the added variable is an uninitialized integer, the default uninitialized variables are set to the value of 0). Secondly, the kernel function that is responsible for reading the ELF header must set the process control block to a special value equal to one (the pseudo code is shown in Algorithm 1). In other words, for the memory management subsystem to detect the wanted process, the process must be marked.

In essence, only a single bit would be needed to mark if the process is special, but since most of the data structures in a process use variable definitions, the same idea was used for the special marking. To conclude, the process is marked at the link level by changing the magic number value, as shown in Figure 6; the kernel can detect this changed magic number value and set the additional process control block integer value equal to 1 at the process level.

```
1 if process magic numbers are equal to 0ELC then
2     set process integer value to 1
```

Algorithm 1: Pseudo Code of Process Marking.

4.4 Memory Decryption Implementation

The core of this work is the run-time decryption of the process. In this section, the implementation of the memory subsystem is described. This section describes four topics: changing the memory subsystem execution flow, initial page fault checks, core logic, adding an additional page flag and macros, and looping through the virtual memory area.

One of the greatest difficulties that arises when trying to implement a new logic into a subsystem is the dilemma of the placement of the new desired logic. In other words, since the memory subsystem function chain is huge, the exact needed location for implementation must be found. If the implementation were to begin at the start of the architecture-independent page fault layer, the

implementation would fail, since the faulted address page table entry (and the 14 successors that follow) would not be mapped yet.

The criteria are for the page table handler to have already been executed, and once the physical pages are loaded and the virtual pages are mapped to the physical frames, then the decryption of the process's physical frames can take place. That is why, after the page table entry handler function, the core logic is implemented as shown in Figure 7.

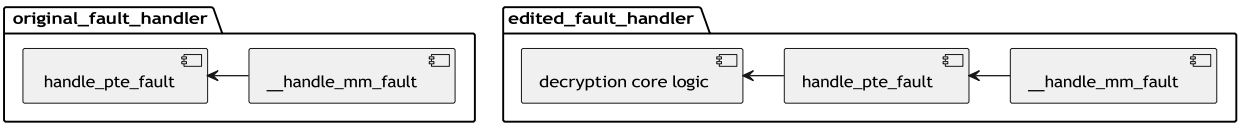

Figure 7. Implementation Location of the Core Decryption Logic in the Memory Subsystem.

4.4.1 Initial Page Fault Checks

The first thing that needs to be checked is if the virtual address of a process that faulted is of the marked special process. Secondly, the flags of the virtual memory area need to be checked. In this case, the text segment is of interest, so the page needs to contain the read and execute flag bits. The necessary checks are implemented in the kernel as shown in Algorithm 2. Lastly, the page table entry mapping needs to be checked; it must contain a valid value.

Linux uses a five-level page table hierarchy; all of the corresponding virtual address offsets need to be checked in order, starting from the PGD and ending with the PTE. If any of these offsets do not yet contain a valid value (are not yet mapped), the function returns the original fault structure and skips the decryption process.

```
1 if current process is marked special then
2     if page flags are read and execute then
3         return success;
4     else
5         return vmFault;
```

Algorithm 2: Inside the Architecture Independent Page Fault Handler. Initial Page Fault checks.

4.4.2 Core Page Decryption Logic

If all the mentioned criteria in the section 4.4.1 match, the function can continue with the core logic implementation. One of the most challenging parts of this thesis project is to retrieve and decrypt only the necessary pages for process execution, meaning those pages that are loaded on demand by the kernel memory subsystem. An easier solution could be implemented by loading all the pages of a process into memory, and then decrypting them one by one, and once all pages are decrypted, set the program counter to the process's first instruction address and begin executing. But this is not possible on modern systems by default and would defeat the whole purpose of virtual memory (enabling the execution of a process that is not fully loaded into memory). It would be possible to implement this by reconfiguring the kernel, but that would immensely slow down the process load time, depending on the size of the text segment.

The program that is being tested (a server) has 119 pages. Decrypting all the pages at run-time before executing the first instruction would take at least 7 to 8 times longer, depending on

the operating system's current state. Since the easier solution is far from optimal, demand page decryption is implemented. But here is another issue: since each page fault address maps to a single PTE, this would logically conclude that a single page fault loads and maps a single physical frame into memory. But that is not the case in the Linux kernel. In the tested environment, the actual number of pages loaded per single page fault is 15 pages. The exact number of pages loaded is architecture-specific; in this case, it reflects how memory page loading is configured for this system. This increases the complexity of the implementation, since there should be a mechanism developed to distinguish between pages that have been decrypted and have not yet been decrypted.

To combat this issue, an additional flag bit was added to the page structure, as shown in Figure 7. In addition to the page flags 3 following macros were added to work with the new page flag bit: `SetPageRamcrypt`, `ClearPageRamcrypt`, `PageRamcrypt`. Each of these corresponds to its functionality in the following order: set the encrypted page bit, clear the encrypted page bit, check if the page has been encrypted.

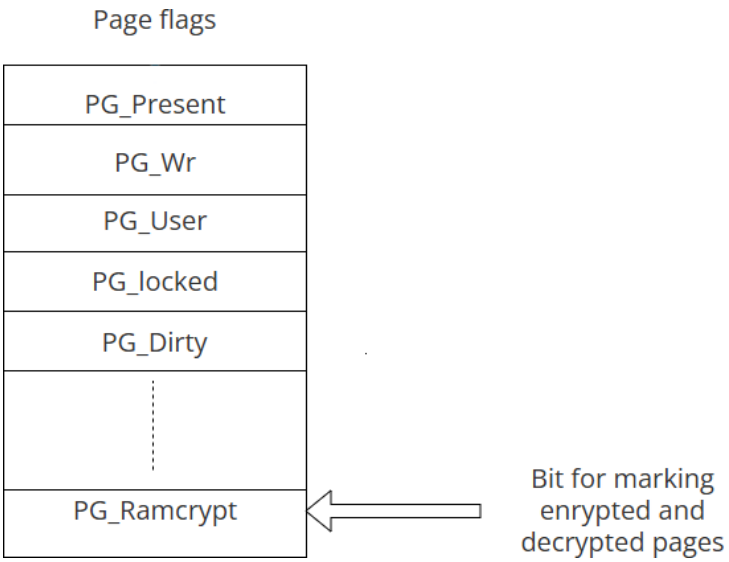

Figure 8. Adding an Additional Bit for Marking Decrypted and Encrypted Pages.

4.4.3 Looping Through the Virtual Memory Area

Once a mechanism was added to differentiate the page encryption state, there should also be a mechanism developed that would detect all 15 mapped pages per page fault. A solution was implemented by looping through all the process virtual memory areas. It is important to loop through all the areas, since the process can jump to any address of its linear address space (to any of its mapped absolute addresses). Once again, since the encrypted area of a process is the text segment, the area of interest is the text segment virtual area. Each cycle increments the loop count by the page size, in this way checking for all the virtual pages in the area. By doing this, every single virtual page can be checked to see if it has been mapped to a physical frame.

In order to check if the virtual page has been mapped to a physical frame (since only then is there a physical frame that can be decrypted), the PTE entry must be retrieved from the virtual address. This is again implemented by the page table walk (from PDG to PTE). From the PTE, the

physical address is retrieved, and finally, the actual physical page structure is retrieved. In this case, it is done by first getting the page frame number, which is then passed to a kernel-built-in function to get the actual page.

At this point, since the actual physical frame is retrieved because it is not null, this is where the implementation of the special page bit and the page macros are extremely useful in differentiating the pages that are decrypted from the pages that still need to be decrypted. By using the macro mentioned in the previous section `PageRamcrypt`, the special bit on the page can be checked to retrieve the decryption state. If it is not decrypted, the page decrypted flag is set using the `SetPageRamcrypt` macro, and then the decryption process can start.

4.4.4 Physical Frame Decryption

The actual page decryption is straightforward with only a few caveats. Before decryption, the page needs to be locked. The page belongs to the text segment and has read and execute flag bits only; locking may seem unnecessary because the process will not modify the page. But here is the catch: the kernel memory subsystem can alter the page structure values at any time, so locking always ensures safety.

Another thing which must be done is to map the page to be edited to a high address using the kernel function `kmap`. This function maps the requested region of memory into 128 MB memory above kernel space, which is especially reserved for tasks like these (the full pseudo code of the mentioned implementation can be seen in Algorithm 3).

```
   Data: virtual memory areas
   Result: decryption of the pages
1  while a virtual memory area of the text segment is not looped through till the last virtual
     page do
2      retrieve the Page Frame number;
3      if Virtual Frame is mapped to a Page Frame then
4          retrieve the Page frame;
5          if Page Frame is not marked decrypted then
6              mark the page as decrypted and decrypt the page
```

Algorithm 3: Main Decryption Logic.

4.5 Asymmetric Key Encryption

One of the goals of this work was to implement the encryption at the kernel level using asymmetric key encryption using the AES encryption algorithm (The Advanced Encryption Standard is a widely used encryption algorithm to protect data and communications in today's digital age [3]). But this was not made possible to implement due to the complexity this method introduces, which requires a lot of time and resources beyond the current capabilities. Here, we discuss the technical difficulty in completing the task.

The effect of asymmetric key encryption is the reduction of data size after encryption. This is especially challenging for the paging system, which the kernel uses, since it operates in fixed-size page chunks. In order to understand the issue, consider an example shown in Figure 9 . Each page fault generates 3 pages to be loaded into memory. As an example for this discussion, let's consider that the compression amount for each page encrypted using asymmetric encryption is 36 percent,

a bit less than half of the page. Since asymmetric encryption compresses the page size, the page fault handler loads five pages rather than three. There is not enough space to decrypt the pages, so the kernel must service an additional two pages.

The biggest difficulty concerning the implementation lies in the fact that all the page table entries need to be completely remapped in the process control block. If this were not done, the program would crash. In addition, corrections need to be made in the ELF executable file. The binary segment variable, which describes the text segment size, needs to be changed to the reduced value corresponding to the actual new text segment size.

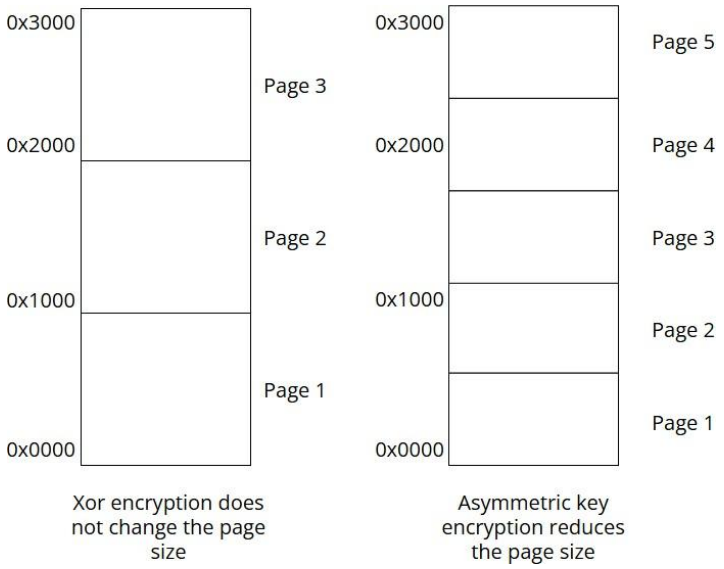

Figure 9. Data and Function Compression Impact on the Process Address Space.

4.6 Sharing the Environment

In essence, there is no difference in what containment method or tool will be used to isolate the encrypted program that will be executed, because it is at the kernel level where the decryption is being performed. For testing purposes, the `chmod` tool was used to isolate the process file access for simplicity. The implementation was successful as predicted, the kernel decrypts the program and runs it, even though it is isolated in a `chroot` file system container. But there is one more unresolved issue left. The goal of this task was also to enable sharing the isolated application across different environments while preserving its decryption functionality. The reason why it was not possible will now be discussed.

Writing a kernel patch for multiple versions is almost impossible because most of the time, data structures and functions are changed or new ones are added to the kernel code base by the kernel developers (in different kernel versions, different variable, structure, and function names are used). This makes the memory decryption solution implemented in the kernel patch available only to the targeted kernel version and only workable on that version. To run it on any other environment or machine, the kernel must be recompiled and installed on a system that supports kernel version 6.12.58.